

金融決済プラットフォーム システム導入提案

GCPメイン+AWS連携による次世代ハイブリッドクラウド基盤
高可用性・拡張性・PCI DSS準拠を実現するモダンアーキテクチャ

 Google Cloud

 AWS Connected

 GKE Autopilot

 PCI DSS Compliant

 99.95% Availability

エグゼクティブサマリー

🎯 プロジェクトの目的

次世代決済基盤の高信頼・高性能化と俊敏な機能追加を実現

- **市場競争力の強化:** 急増するデジタル決済需要に対応するスケーラビリティの確保
- **ビジネス俊敏性:** マイクロサービス化による新機能の即時投入体制の確立
- **信頼性向上:** 金融グレードのセキュリティと99.99%を目指す可用性

⚠️ 課題と解決策

CURRENT ISSUES

- × レガシーな性能限界
- × 運用負荷の増大
- × リリースの長期化



ONNS

Autopilot採用

Ops自動化

マイクロサービス化

📈 期待される効果

5倍

処理能力向上

30%

運用コスト削減

3日

リリースサイクル (以前は2週間)

99.95%

システム可用性保証

💰 投資対効果 (ROI)

12ヶ月 で投資回収見込み

- **CAPEX最小化:** フルマネージドサービス活用による初期構築工数の削減
- **OPEX最適化:** Committed Use Discounts (CUD) とオートスケーリングによる無駄の排除
- **機会損失防止:** 高可用性によるダウンタイム損失の回避

プロジェクト概要・非機能要件



処理能力 (THROUGHPUT)

10,000 TPS

ピーク時のスパイクアクセスにも耐えうる
高スループット設計



可用性 (AVAILABILITY)

99.95 %

マルチゾーン構成とRegional HAによる
ダウンタイム極小化



セキュリティ & コンプライアンス

✓ PCI DSS Level 1 準拠

- ✓ **データ保護:** カード情報のトークン化と、保存時・転送時の強力な暗号化 (Cloud KMS / TLS 1.3)
- ✓ **アクセス制御:** 最小権限の原則に基づく IAM と Workload Identity による厳格な権限管理
- ✓ **監査証拠:** 全操作のログ記録と変更不可な監査ログ保管 (SIEM 連携)



レイテンシ (LATENCY)

< 200 ms

P95 (95パーセンタイル) 値。
ユーザー体験を損なわない高速レスポンス



拡張性 (SCALABILITY)

Auto Scaling

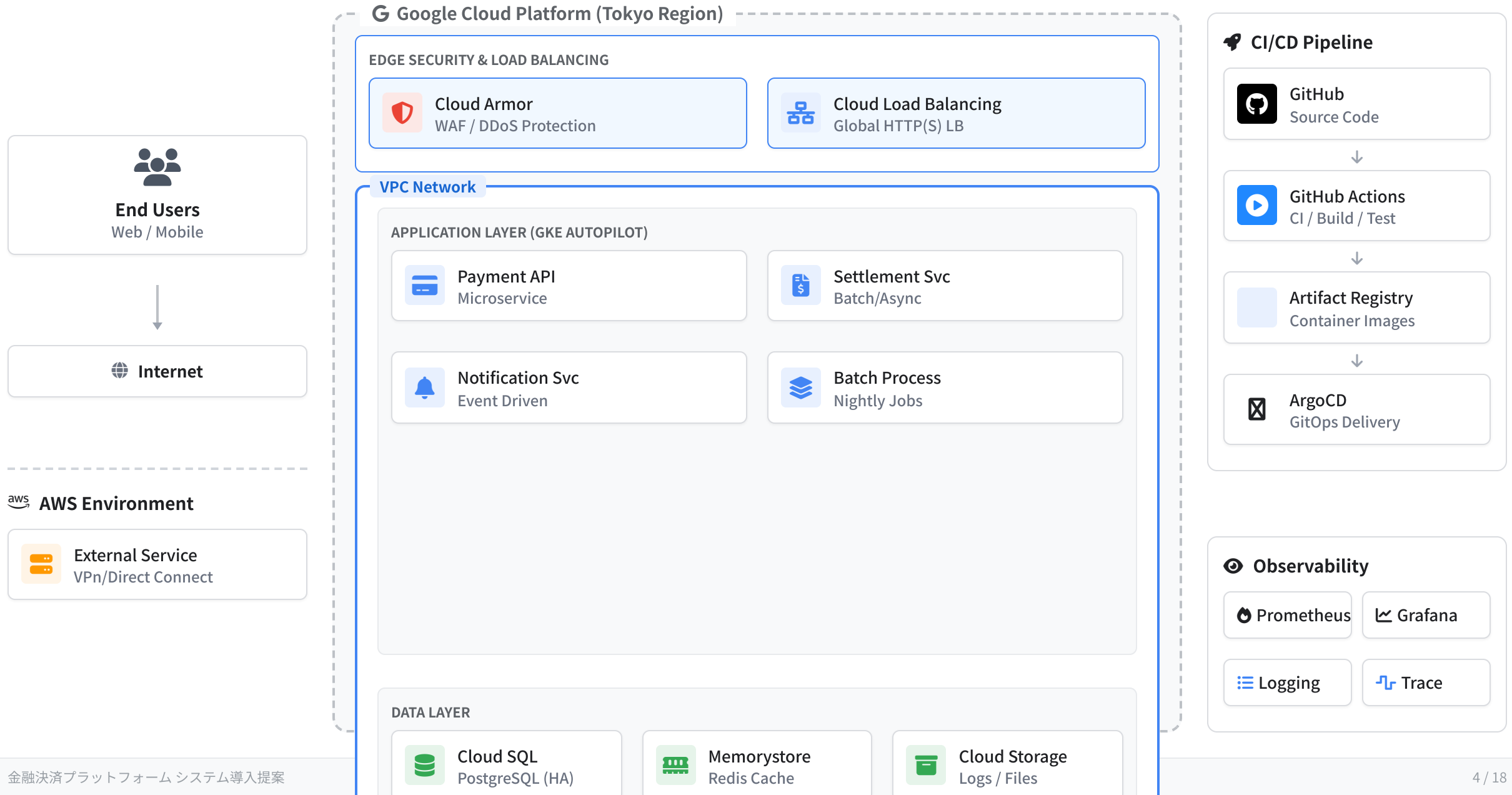
HPA / VPA によるポッド自動増減と
GKE Autopilot によるノード自動管理



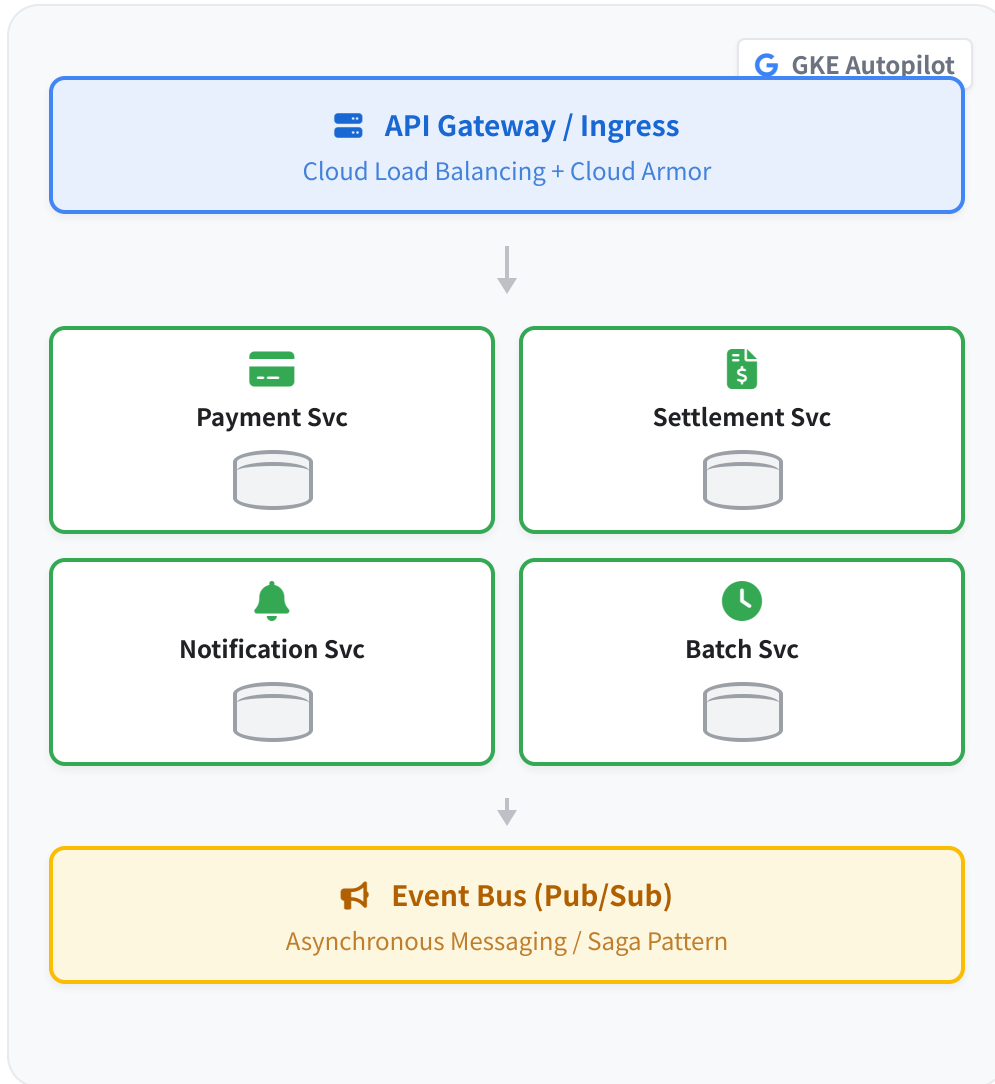
技術標準・運用要件

- ☁ **クラウドネイティブ:** コンテナ (GKE) とマネージドサービス (Cloud SQL) を全面的に採用
- 🔗 **GitOps運用:** ArgoCD による宣言的なインフラ・アプリ管理と自動デプロイ
- 👁 **可観測性 (Observability):** メトリクス・ログ・トレースの3本柱によるシステム状態の完全な把握

システム全体構成図 (GCP + AWS)



技術アーキテクチャ概要



マイクロサービスアーキテクチャ

ドメイン境界（決済、清算、通知等）に基づく疎結合なサービス分割。
チームごとの独立したデプロイとスケーリングを実現。



イベント駆動 (Event-Driven) PUB/SUB

サービス間の連携を非同期メッセージングで実現。
Sagaパターンによる分散トランザクションで、システム全体の結果整合性を担保。



API-First デザイン OPENAPI

OpenAPI仕様書をSSOT（信頼できる唯一の情報源）として定義。
クライアントとサーバー間の契約を明確化し、並行開発を促進。



高信頼性デザインパターン

冪等性 (Idempotency): リトライ時の二重処理を防止。
サーキットブレーカー: 障害の連鎖を防ぎ、システム全体の安定性を維持。

技術スタック詳細



CONTAINER PLATFORM

GKE Autopilot

- **NoOps運用:** ノード管理・アップグレードが完全自動化され、運用工数を大幅削減
- **セキュリティ標準化:** Google推奨のセキュリティベストプラクティスがデフォルト適用
- **従量課金:** Podのリソース要求量に基づく課金で、オーバープロビジョニングを排除



RELATIONAL DATABASE

Cloud SQL (PostgreSQL 15)

- **高可用性 (HA):** マルチゾーン構成による自動フェイルオーバーでダウンタイムを最小化
- **データ保護:** ポイントインタイムリカバリ（35日保持）と自動バックアップ
- **セキュリティ:** IAM認証連携と透過的なデータ暗号化（TDE）



IN-MEMORY CACHE

Memorystore (Redis 7.0)

- **高速レスポンス:** サブミリ秒のレイテンシでDB負荷を軽減し、API性能を向上
- **HA構成:** レプリカ自動昇格による高可用性でセッション情報を保護
- **スケーラビリティ:** ダウンタイムなしでの容量拡張とバージョンアップ



OBSERVABILITY

Prometheus / Grafana

- **統合監視:** インフラからアプリまで全レイヤーのメトリクスを一元管理
- **SREダッシュボード:** SLI/SLOやエラーバジェットを可視化し、健全性を常時監視
- **Cloud Monitoring連携:** GCPマネージドサービス（Google Cloud Managed Service for Prometheus）を活用



CI/CD & GITOPS

GitHub Actions / ArgoCD

- **自動化パイプライン:** Lint、テスト、セキュリティスキャン、ビルドを完全自動化
- **GitOps:** インフラとアプリの状態をGitで宣言的に管理し、ArgoCDで自動同期
- **安全なデプロイ:** Canaryリリースや自動ロールバック機能によるリスク低減



INFRASTRUCTURE AS CODE

Terraform

- **コード管理:** 全リソースをコード化し、バージョン管理とレビュープロセスを適用
- **再現性:** 環境（本番、Staging、開発）間の構成差異を排除し、迅速な環境構築を実現
- **モジュール化:** 再利用可能なモジュール設計で、標準化とガバナンスを強化

マイクロサービス構成

MICROSERVICES LAYER



Payment API

決済承認、オーソリ、キャプチャ処理を担当する同期型API。高可用性が求められるコアサービス。

Go gRPC PostgreSQL



Settlement Svc

加盟店への入出金、振替処理を行う非同期サービス。結果整合性を担保。

Java Spring Boot Cloud SQL



Notification Svc

メール、Push通知、外部Webhook送信を一元管理するイベント駆動サービス。

Node.js Pub/Sub Redis



Batch Processing

日次締め処理、帳票作成、データアーカイブを実行するスケジュールジョブ。

Python Cloud Run Jobs BigQuery



認証基盤 (OIDC)



共通セキュリティ



監査ログ (Audit)



Config / Secret管理



データ整合性パターン



Outbox Pattern:

DB更新とイベント発行を同一トランザクションで行い、メッセージロストを防止。



Saga Pattern:

分散トランザクションを補償トランザクションの連鎖で管理し、整合性を維持。



Idempotency Key:

リクエストに一意的なキーを付与し、ネットワーク障害時の二重処理を確実に防止。



信頼性・運用設計



Circuit Breaker:

障害発生サービスへの呼び出しを遮断し、システム全体の共倒れを回避。



Retry with Backoff:

一時的なエラーに対して、指数バックオフを用いた適切なリトライを実施。



Contract Testing:

コンシューマ主導契約テストにより、サービス間のインターフェース整合性を担保。

セキュリティとコンプライアンス (PCI DSS)



Layer 1: 境界防御 (Perimeter)



Cloud Armor (WAF/DDoS)



Geo-IP Restriction



TLS 1.3 Termination



Layer 2: ネットワーク分離 (Network)



VPC Separation



Private GKE Cluster



Egress Filtering



DEFENSE IN DEPTH



Layer 3: アクセス制御・コンピュータ (Identity)



Cloud IAM (Least Privilege)



Workload Identity



Binary Authorization



Layer 4: データ保護 (Data)



Secret Manager



Data Tokenization



Cloud KMS (CMEK)

PCI DSS v4.0 Compliant

Level 1 Service Provider

🛡️ ネットワークセキュリティの構築と維持

- ✓ Req 1: NSG/Firewallの設定と維持
- ✓ Req 2: ベンダーデフォルト値の不使用

🔒 口座データの保護

- ✓ Req 3: 保存データの保護 (トークン化)
- ✓ Req 4: 伝送データの強力な暗号化 (TLS 1.3)

🔍 脆弱性管理プログラムの維持

- ✓ Req 5: マルウェア対策 (Container Scanning)
- ✓ Req 6: 安全なシステム開発 (DevSecOps)

👤 強力なアクセス制御手法の導入

- ✓ Req 7: 業務上の必要性に応じたアクセス制限
- ✓ Req 8: ユーザーの識別と認証 (MFA)
- ✓ Req 9: データへの物理アクセスの制限 (GCP)

🔍 ネットワークの定期的な監視・テスト

- ✓ Req 10: ネットワークリソースへのアクセス追跡
- ✓ Req 11: セキュリティシステムの定期テスト

暗号化とデータ保護



保存時暗号化 (At Rest)

Cloud KMS



CMEK (Customer Managed Encryption Keys):

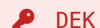
顧客管理の暗号鍵を使用し、Googleによるアクセスも制御。Cloud SQL, Storage, Memorystoreなど全データストアに適用。



自動鍵ローテーション:

90日ごとの鍵自動更新により、万が一の鍵漏洩時のリスクを最小化。

Plain Text



DEK



KEK



Encrypted



転送時暗号化 (In Transit)

TLS 1.3 / mTLS



TLS 1.3 強制:

最新のTLSプロトコルのみを許可し、古い脆弱な暗号スイートを無効化。HSTSによりHTTPS接続を強制。



mTLS (Service Mesh):

マイクロサービス間の通信を相互TLSで暗号化・認証。内部ネットワークでの盗聴やなりすましを防止。

Client



TLS 1.3

LB / WAF



mTLS

Pod



カード情報保護

PCI DSS Req 3



PAN非保持化 (Tokenization):

カード番号(PAN)を即座にランダムなトークンへ変換。システム内部ではトークンのみを流通・保存し、漏洩リスクを根絶。



DLP (Data Loss Prevention):

ログやストレージを定期スキャンし、誤って保存された機密情報を自動検知・マスキング。



User

PAN

Token Svc

Token

App / DB



バックアップ暗号化

DR / BCP



Cloud SQL Backup:

自動バックアップデータもCMEKで暗号化。リージョン間転送時もGoogleのバックボーンネットワーク内で暗号化を維持。



署名付きURL (Signed URL):

バックアップファイルへのアクセスは、期限付きの署名付きURLでのみ許可し、アクセス権限を厳格に管理。

Primary

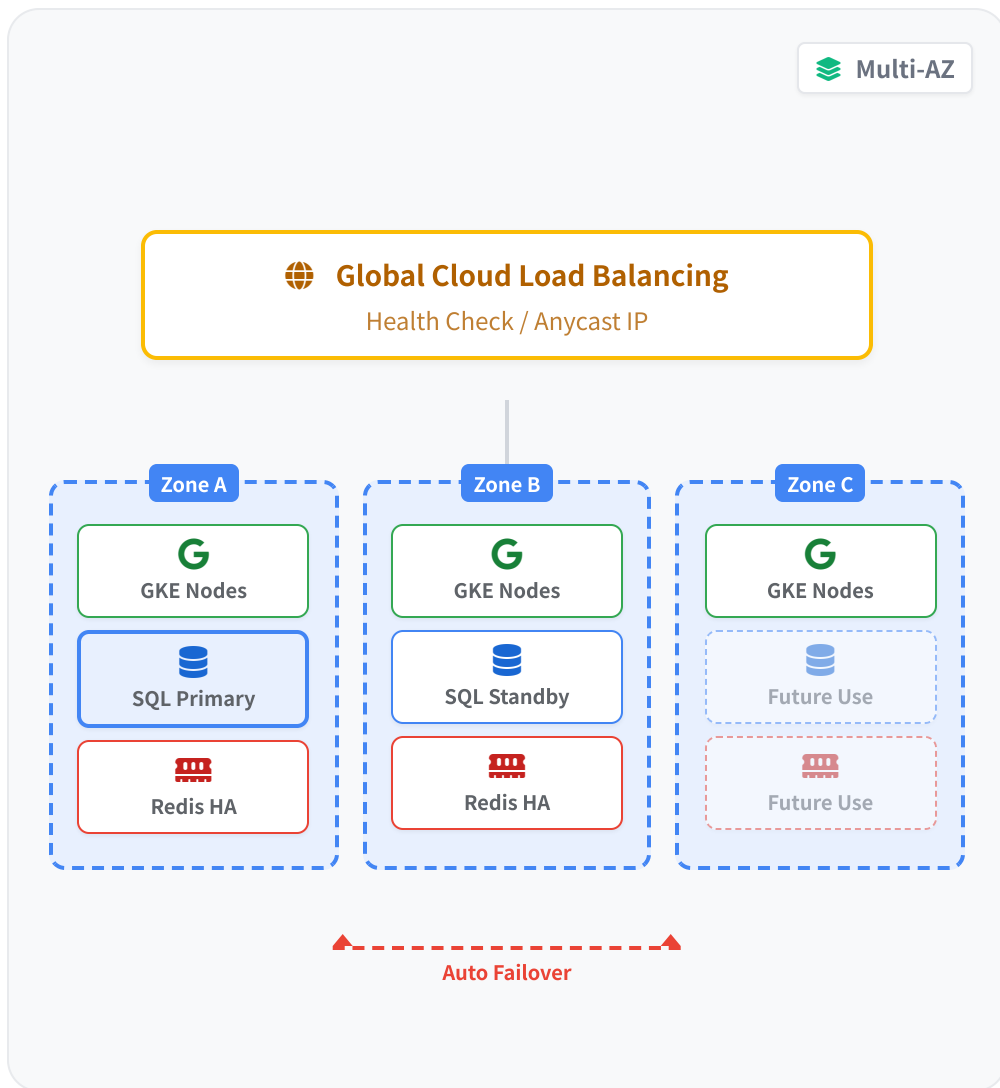


Encrypted



Backup

高可用性設計



コンピューットの冗長化

GKE AUTOPILOT

マルチゾーン構成で単一ゾーン障害を回避。

Pod Disruption Budget (PDB)により、メンテナンス時も最低稼働数を保証。



データ層の高可用性

REGIONAL HA

Cloud SQL: プライマリ障害時にスタンバイへ自動フェイルオーバー。

Memorystore: Redis HA構成でキャッシュ層も冗長化し、高速な復旧を実現。



ネットワーク・レジリエンス

Global Load Balancingによる負荷分散とヘルスチェック。

異常なバックエンドを即座に切り離し、健全なインスタンスのみにルーティング。



ゼロダウンタイム設計

ステートレスなアプリケーション設計とセッション外部化 (Redis)。

ローリングアップデートにより、サービスを停止せずにデプロイ可能。

ディザスタリカバリ（DR）戦略

🕒 RECOVERY TIME OBJECTIVE

4時間

システム復旧までの最大許容時間。
ArgoCDによる迅速なインフラ再構築で実現。

💾 RECOVERY POINT OBJECTIVE

15分

データ損失の最大許容範囲。
Cloud SQLのトランザクションログ同期で担保。

🔄 DR訓練 / リハーサル

年2回

半期ごとに災害シミュレーションを実施。
Runbookの有効性を定期検証。



運用体制とSRE

Service Level Objectives (SLO)

可用性 (Availability) **99.95%**

SLI: (成功リクエスト数 / 全リクエスト数) * 100

レイテンシ (Latency) **P95 < 200ms**

SLI: 200ms以内に完了したリクエストの割合

エラー率 (Error Rate) **< 0.1%**

SLI: 5xxエラーの割合

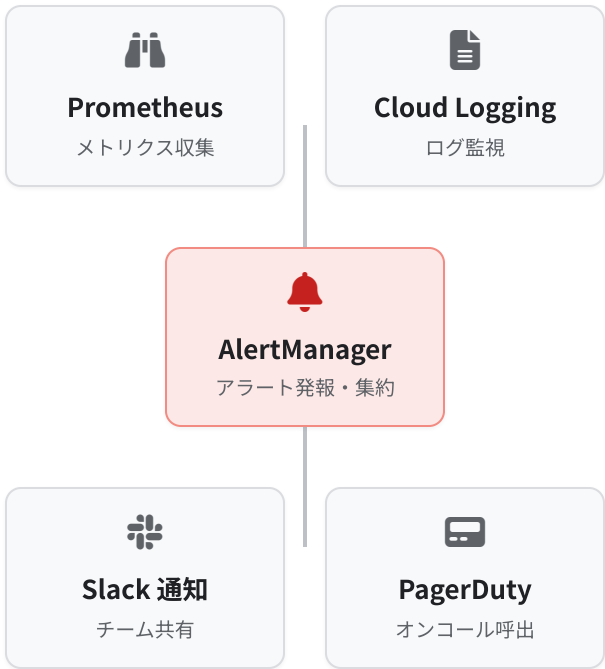
エラーバジェット

100% - SLOの残余分を「予算」として管理。予算内であれば新機能リリースを優先し、枯渇時は信頼性向上タスクに注力する判断基準として活用。

ポストモーテム

障害発生後は「非難なき振り返り (Blameless Postmortem)」を実施。根本原因の特定と再発防止策の策定に焦点を当て、ナレッジとして共有。

監視・運用フロー (24/365)



SRE Team

プラットフォーム健全性維持、自動化ツール開発、SLO管理

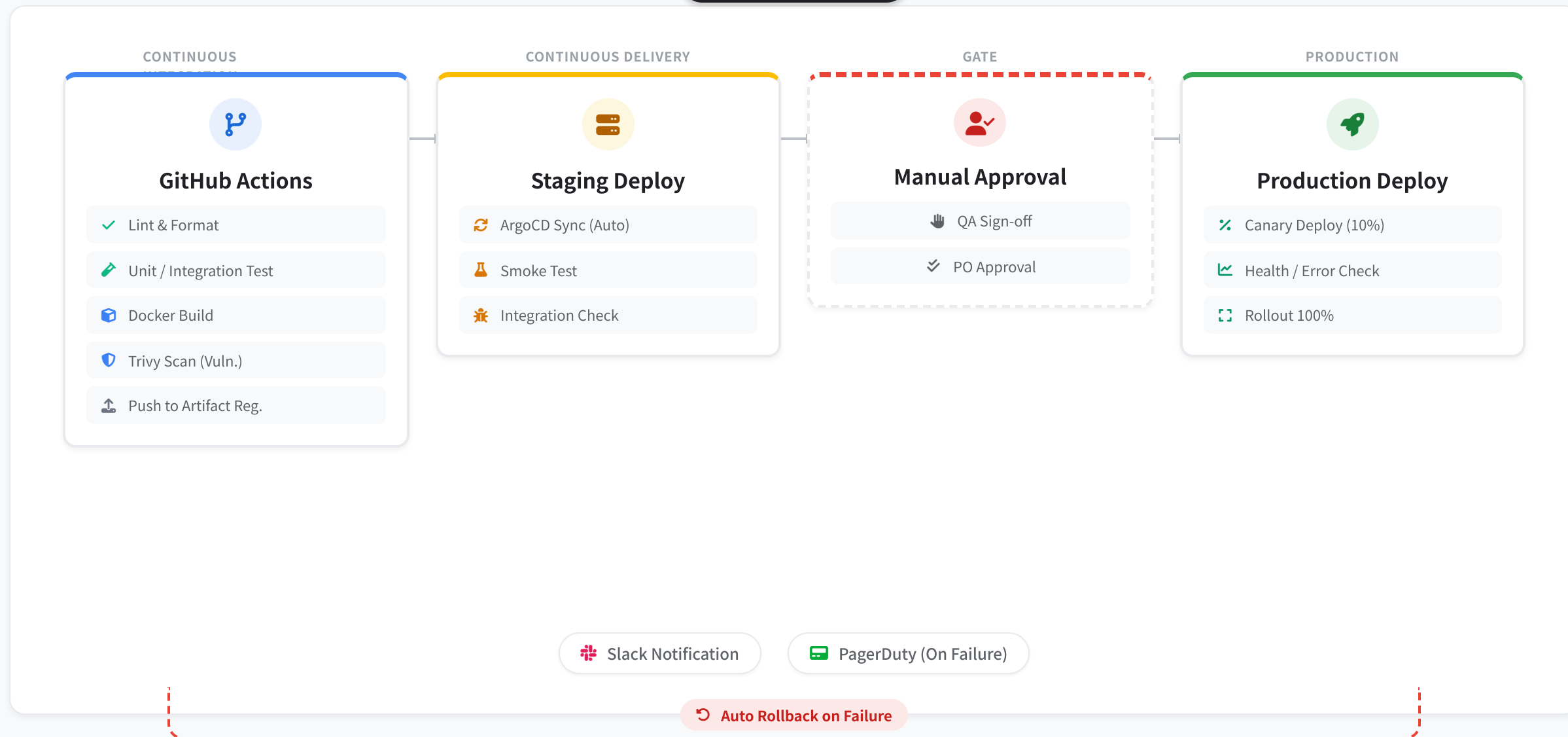


Dev Team

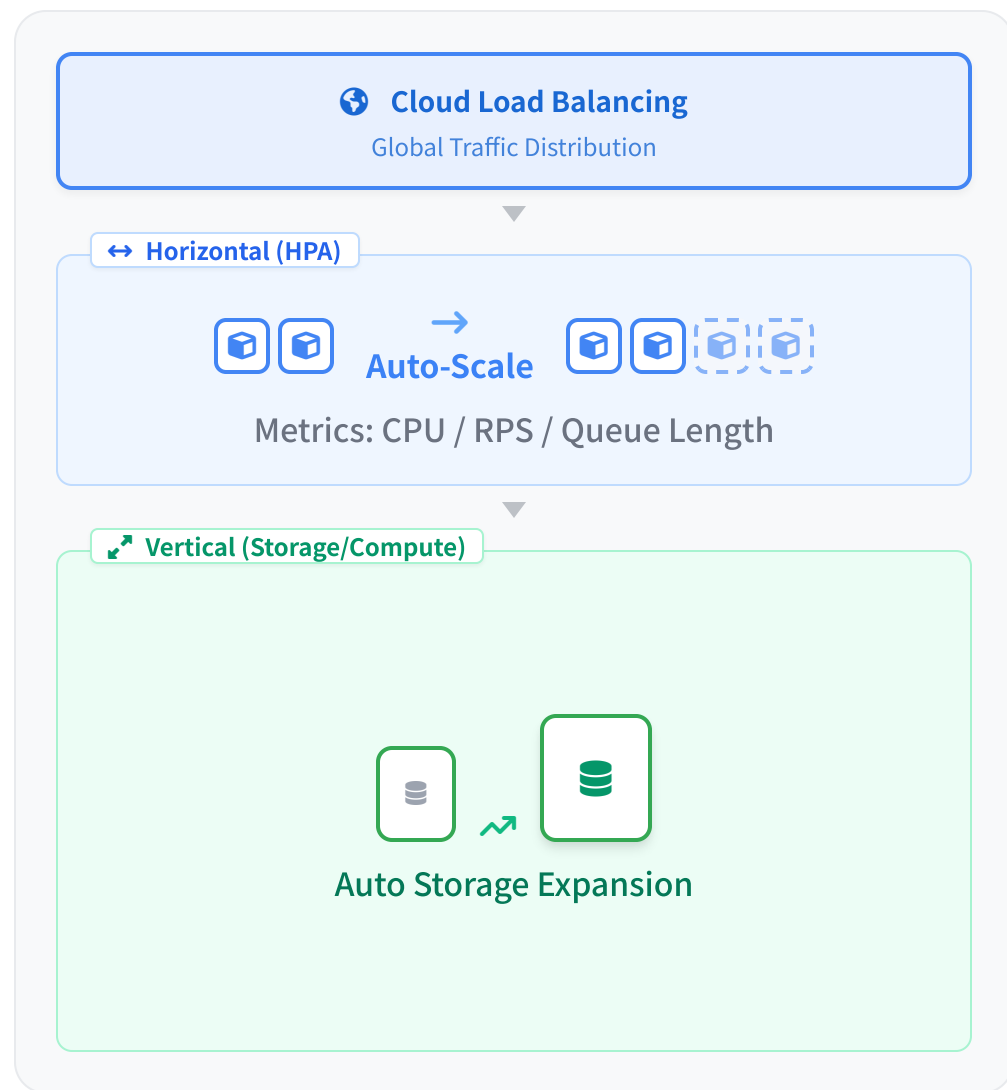
アプリ機能開発、オンコール（二次対応）、ポストモーテム参加

CI/CDパイプライン (GitOps)

GitOps Workflow



スケーラビリティ設計



水平スケーリング HPA

CPU使用率、RPS、キュー長をトリガーにPod数を自動増減。
急激なトラフィックスパイクにも即座に対応し、可用性を維持。



垂直スケーリング VPA / AUTO-GROW

Cloud SQLやMemorystoreのストレージ容量を自動拡張。
GKE Autopilotにより、Podのリソース要件を最適に調整。



負荷分散とトラフィック制御

Global Load Balancingによる地理的な負荷分散。
Cloud Armorを用いたレートリミットでシステム過負荷を防止。



パフォーマンス最適化

キャッシュ戦略の最適化でDB負荷を軽減し、レイテンシを改善。
非同期処理とN+1問題解消により、スループットを最大化。

コスト見積もり（月額）

月額総計 (推定)

\$3,880

全環境合計

年間コスト (推定)

\$46,560

月額 × 12ヶ月

環境数

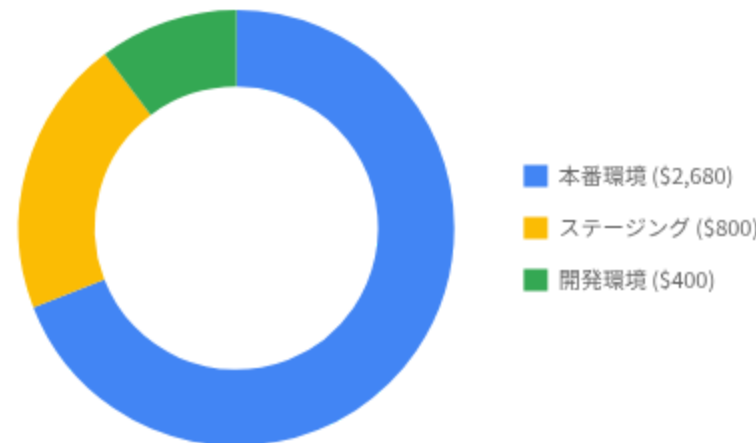
3

Prod / Stg / Dev

本番環境コスト内訳 (Production)

Service Component	Config / Spec	Monthly Cost
 GKE Autopilot	Node管理不要 / Auto-scaling	\$1,200
 Cloud SQL (PostgreSQL)	db-n1-highmem-4 x 2 (HA)	\$800
 Memorystore (Redis)	5GB Standard HA	\$300
 Cloud Load Balancing	Global HTTP(S) LB	\$100
 Cloud Storage	1TB Standard Class	\$30
 Observability	Logging / Monitoring	\$200
 Artifact Registry	Container Images	\$50
Total (Production)		\$2,680

環境別コスト配分



コスト最適化施策

Committed Use Discounts (CUD)

1年または3年の確約利用割引を適用

20-30% OFF

開発環境の自動停止

夜間・週末のCloud SQL/GKE停止 (Cloud Scheduler)

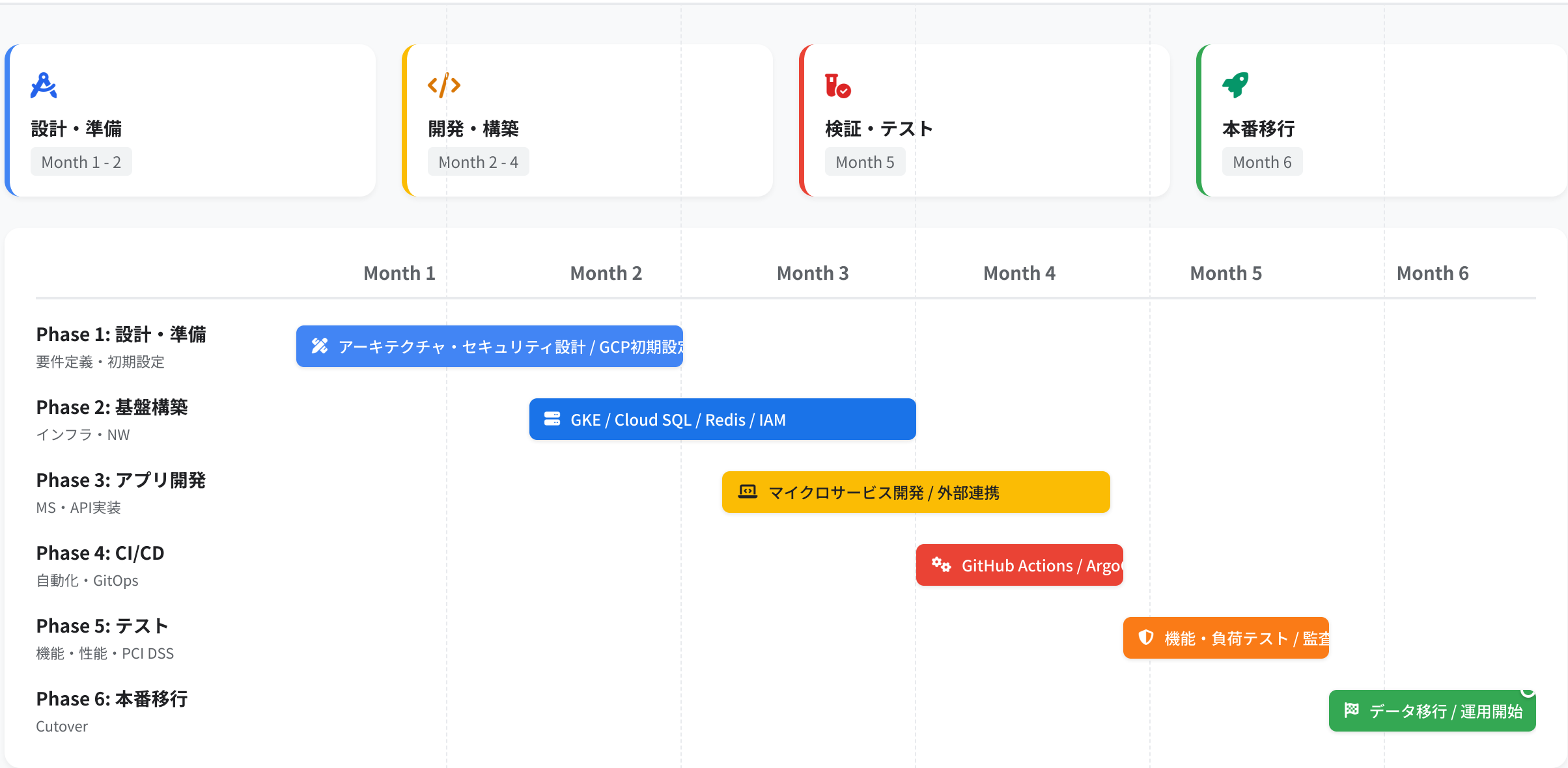
40% OFF

リソース適正化 (Right-sizing)

VPAによるPodリソース最適化とインスタンス調整

15% OFF

導入スケジュール（6ヶ月計画）



まとめ・次のアクション



次世代金融決済プラットフォーム基盤の実現に向けて

GKE Autopilotとマイクロサービスアーキテクチャの採用により、「**高可用性・高性能**」と「**俊敏性・拡張性**」を両立。PCI DSS準拠の堅牢なセキュリティと、SREによる安定運用体制を提供します。



次の一手 (Next Steps)

- ☒ **提案・概算見積提示**
本資料による方針合意
- ☐ **PoC (概念実証) 開始**
2〜4週間、技術的実現性の検証
- ☐ **詳細設計フェーズ**
パラメータ設計、詳細移行計画
- ☐ **構築・実装着手**
Phase 2 基盤構築へ移行



意思決定事項 (Decisions)

- ☐ **RTO / RPO 目標値の合意**
RTO 4h / RPO 15min で承認可否
- ☐ **DRリージョン選定**
大阪リージョンの採用承認
- ☐ **CUD (確約利用割引) 適用**
1年契約によるコスト削減の承認



準備物 (Preparation)

- ☐ **既存API仕様書**
OpenAPI (Swagger) 定義ファイル
- ☐ **ドメインモデリング資料**
マイクロサービス境界の定義
- ☐ **監査要件リスト**
QSAからの指摘事項・要件一覧
- ☐ **プロジェクト体制図**
各社担当者アサイン表



まずはPoCスコープ定義のお打ち合わせをお願いいたします。

Schedule Meeting